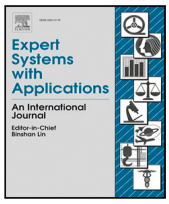
Expert Systems With Applications 284 (2025) 127773

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa



Mathematical formulations for the scheduling of unrelated parallel machines considering machine availability and eligibility

Alejandro Uribe [a,b,*], Urtzi Otamendi [b,c], Arkaitz Artetxe [b], Juan Carlos Rivera [a]

[a] School of Applied Sciences and Engineering, Universidad EAFIT, Carrera 49 N 7 Sur-50, Medellín 05001, Antioquia, Colombia
[b] Vicomtech Foundation, Basque Research and Technology Alliance (BRTA), Donostia-San, Sebastián, 20009, Spain
[c] Department of Computer Sciences and Artificial Intelligence, University of the Basque Country (UPV/EHU), Donostia-San, Sebastián, 20009, Spain

ARTICLE INFO

Keywords:
UPM
Scheduling
Shifts
Makespan
Mixed integer linear programming

ABSTRACT

Job scheduling in manufacturing companies faces potential challenges due to varying job durations per machine, working calendar structures, and preventive maintenance. This must be considered to avoid unplanned outages and respect planned downtime. This research contrasts two literature-derived models with a novel approximation to solve the Unrelated Parallel Machine scheduling problem (UPM), incorporating machine availability and eligibility constraints. The implemented models address the optimization of two major metrics: the makespan, and the completion time of the last shift used. Minimizing these measurements is crucial to maximize system efficiency and reduce high-demand industries' operating costs. The models differ in their structure and how they were conceptualized. Model 1 uses sequence-dependent variables, offering detailed job sequencing but increasing problem size as job counts rise. Model 2 simplifies by assigning jobs to machines and shifts without sequencing, improving computational efficiency but compromising detail. Model 3, the novel proposal of this research, balances detail and efficiency by incorporating start-time-dependent variables, suitable for large-scale problems. Tests were performed varying the number of jobs, machines, shift lengths, and mean job durations. The results show that Models 2 and 3 outperform at makespan minimization, obtaining an optimal solution to almost all benchmark instances within an hour. Model 1 struggles with scalability, achieving less than 0.5% GAP in 69 out of 96 small instances and failing for larger setups. For the cases where the minimization of the completion time of the last operating shift is considered, all models improve computational times and GAPs. These findings guide the selection of scheduling models based on available computational resources and detailed planning needs in high-demand industrial environments.

1. Introduction

Manufacturing companies are becoming more receptive to implementing strategies that improve the performance of their systems in response to accelerated technological advances, new consumer demands, increased market competitiveness, and high labor, material, and energy costs (Shahvari & Logendran, 2017a). In this context, an efficient scheduling and sequencing process plays a crucial role in operations planning, increasing profits, reducing costs, and meeting user expectations (Afzalirad & Rezaeian, 2017). In addition, as industries consume nearly half of the world's energy, and global demand is expected to grow by 37% by 2040, optimizing resource allocation becomes essential to reduce energy consumption and mitigate harmful environmental effects (Sanati, Moslehi, & Reisi-Nafchi, 2023). While adopting more energy-efficient machinery is a viable option, it often requires substantial investments and long-term planning. In contrast, intelligent scheduling strategies provide a flexible and immediate solution to improve production efficiency, minimize waste, and lower operational costs (Chen, Chu, Sahli, & Li, 2024). By effectively managing machine availability and task allocation, scheduling not only enhances throughput but also supports sustainable development, helping industries balance economic performance with environmental and social responsibility (Ezugwu, 2024).

Generally, job scheduling focuses on assigning jobs to machines to optimize a defined performance metric (Abed & Mohmad Kahar, 2023). Job allocation on parallel machines is a common practice in industries such as semiconductor manufacturing and metal production. However, due to the interdependence between jobs and machines, the correct distribution of jobs represents a key challenge in scheduling operations (Ying, Pourhejazy, & Huang, 2024). A specific problem that arises in this context is the UPM, where scheduling requires certain

* Corresponding author at: Vicomtech Foundation, Basque Research and Technology Alliance (BRTA), Donostia-San, Sebastián, 20009, Spain.
E-mail addresses: auribev1@eafit.edu.co, auribe@vicomtech.org (A. Uribe), uotamendi@vicomtech.org (U. Otamendi), aartetxe@vicomtech.org (A. Artetxe), jrivera6@eafit.edu.co (J.C. Rivera).

https://doi.org/10.1016/j.eswa.2025.127773
Received 2 December 2024; Received in revised form 27 March 2025; Accepted 15 April 2025
Available online 5 May 2025

conditions to be met: each job must be assigned to a single machine, each machine can only process one job at a time, jobs cannot be interrupted, and processing time can vary depending on the machine (Abed & Mohmad Kahar, 2023). Efficient management of these elements in operations planning is essential to optimize resources and maximize system performance.

Optimal job allocation in production systems becomes even more complex when constraints such as machine availability and machine eligibility are considered. Machine availability is a critical factor, as machines may be inoperative during certain periods due to preventive maintenance, tool replacement, material shortages, failures, or the end of a work shift (Jaykrishnan & Levin, 2024; Kurt & Cetinkaya, 2024). In traditional machine scheduling, it is often assumed that all machines remain available throughout the scheduling period. However, this assumption does not hold in real-world industrial and manufacturing environments, where machines frequently require downtime for various reasons (Wu & Zheng, 2023). Failing to account for these periods of unavailability can lead to unrealistic scheduling plans and increased disruptions. One of the key reasons to incorporate machine availability into scheduling models is that planned inactivity — such as scheduled maintenance — can prevent larger operational issues. Machines that operate continuously without scheduled downtime are more prone to unexpected failures, which result in longer and more costly interruptions. By proactively managing machine availability, companies can reduce unforeseen breakdowns and ensure a smoother scheduling process (Wu & Zheng, 2023).

On the other hand, machine eligibility refers to the ability of specific machines to process certain jobs based on their technological configuration (Khadivi, Abbasi, Charter, & Najjaran, 2024). Some jobs require specialized equipment due to their technical complexity or size, meaning that not all machines can process them. This constraint is particularly relevant in industries such as medicine, shipbuilding, metallurgy, pharmaceuticals, transportation, and agribusiness, where the specific characteristics of each process, along with machine and personnel operating shifts, make achieving optimal scheduling more challenging (Santoro & Junqueira, 2023). Considering both attributes — availability and eligibility — not only enhances operational efficiency but also enables the development of realistic and robust scheduling strategies. By incorporating planned downtime into scheduling models, companies can proactively mitigate risks and reduce costly last-minute adjustments, leading to a more stable and efficient production system.

This research explores the UPM scheduling problem with machine availability and eligibility constraints. These variations have been considered in other scheduling problems (Aggoune, 2004; Ploydanai & Mungwattana, 2010; Su & Hsiao, 2015). However, for this specific UPM context, the simultaneous consideration of both constraints is relatively novel and has been scarcely explored in the literature. This gap highlights the need for further investigation to develop more effective approaches that account for machine availability windows and eligibility limitations concurrently. To address this gap, two models adapted from the literature are implemented for comparative purposes, and a new model is proposed to enhance existing approaches by providing greater detail and improving computational efficiency. The latter model is inspired by a constraint from the literature, leading to a novel approach that has neither been conceptualized nor implemented for the UPM. Additionally, we introduce modifications to minimize the completion time of the last shift used — referred to as last shift minimization — an objective function closely related to makespan minimization, with better convergence properties and yet unexplored in the literature for this problem. Moreover, in our formulations, single-shift job completion is a requirement.

Minimizing working shifts is a strategic objective that can drive operational efficiency and financial sustainability for companies. Optimizing the number of shifts helps reduce production costs without compromising productivity, directly influencing profit margins and overall cost structures. Carefully selecting the optimal shift configuration is crucial for effective production planning, particularly in manufacturing environments where labor costs represent a significant portion of expenses (Paul & Azeem, 2010). Minimizing shifts not only reduces operational costs — which are often proportional to the number of working days — but also enhances employee well-being by promoting a healthier work-life balance. According to Barck-Holst, Nilsonne, Åkerstedt, and Hellgren (2017), fewer working hours increase satisfaction, lowering turnover rates and cutting recruitment and training expenses. This dual benefit boosts individual performance and drives long-term productivity, making shift minimization a strategic objective for companies pursuing sustainable competitive advantages.

The paper is organized as follows: Section 2 reviews the proposals for the UPM found in the literature. Section 3 presents the proposed mathematical formulations. Section 4 presents the experimental results and analyzes tradeoffs between models and objectives. Finally, Section 5 presents conclusions and future research directions.

2. Literature review

To simplify the analysis of the existing state of the art, we identified common attributes of the jobs, timing, resources, machines, objectives, and solution methods in the different variants and approaches of the UPM scheduling problem. This information is summarized in Table 1. Similarly, these features are further described below.

- **Instance size**: maximum number of jobs N and machines M considered in benchmark instances.
- **Job features**: whether they consider an acceptance or rejection criterion for each job, a release or due date, precedences between jobs, and working by batches or with time windows.
- **Times attributes**: setup times both machine and sequence-dependent.
- **Resources**: whether they use resources such as personnel, material, inventory, or common servers, among others.
- **Machines**: if they consider availability, eligibility, and different machine speeds.
- **Objective function**: if they consider multiple objectives, minimization or maximization.
- **Solution methods**: if they consider exact approaches or methods based on heuristics and metaheuristics.

Approximately 62% of the reviewed articles consider setup times between jobs, either sequence-dependent or machine-dependent. On the other hand, only a few articles consider resources (such as material and operators). Among these, the work by Shahvari and Logendran (2017a) that considers operators with expertise (multi-skill approach) is highlighted. To solve the UPM problem considering batches, the authors proposed a lexicographic approach with mixed-integer linear programming (MILP) formulations and a PSO metaheuristic to minimize time and costs. Another remarkable attribute is the job splitting (Zheng et al., 2022). In this paper, unrelated machines can be rented to process orders through a cloud manufacturing services platform. Here, a uniform processing cost is generated by renting machines for a certain period to manage orders and these can be divided into smaller orders that can be processed simultaneously.

Concerning machine attributes, some approaches use an additional support resource called “common server” where jobs can be occasionally taken to distribute the load (Bektur & Saraç, 2019; Berthier et al., 2022; Raboudi et al., 2022). Similarly, Muñoz-Díaz et al. (2024) proposed using support machines to partially assist other machines in their jobs and then handing them over and running at full capacity. On the other hand, Elyasi et al. (2024) proposed a competitive imperialistic algorithm to minimize the tardiness and earliness of the jobs considering a load-balancing attribute between machines. One of the most interesting papers for this research is that of Santoro and Junqueira (2023) which considers the availability and eligibility of machines with

Table 1
Attributes and methods for solving the UPM.

Cite	Ins		Jobs					ST	Res	Machine			Objective function	Solution method
	N	M	A/R	D	P	B	TW			Av	El	Sp		
Joo and Kim (2015)	100	10						X			X		min makespan	MILP, Hybrid GA
Afzalirad and Rezaeian (2016)	60	8		X	X			X	X		X		min makespan	MILP, GA, Artificial Immune System
Jiang and Tan (2016)			X							X			min makespan, costs	MILP, Heuristic
Shahvari and Logendran (2017a)	100	6		X		X		X	X	X	X		min makespan, costs	PSO, Lexicographic MILP
Afzalirad and Rezaeian (2017)	50	10		X	X			X			X		min flow time, tardiness	MIP, Multi-Objective Ant Colony Optimization, NSGAII
Fanjul-Peyro, Perea, and Ruiz (2017)	30	6							X				min makespan	MILP, Matheuristic
Avdeenko, Mezentsev, and Estraykh (2017)	70	10		X					X				max profit	Dynamic Programming, Heuristic
Shahvari and Logendran (2017b)	26	6		X		X		X					min tardiness, makespan	MILP, Tabu search
Wang and Alidaee (2019)	3000	100											min makespan, makespan	Tabu Search
Fanjul-Peyro, Ruiz, and Perea (2019)	1000	8						X					min makespan	MILP, Branch & Check
Perez-Gonzalez, Fernandez-Viagas, Zamora García, and Framinan (2019)	250	30		X				X			X		min tardiness	MILP, GRASP, Variable Neighborhood Descent
Bektur and Saraç (2019)	100	10		X				X	X		X		min tardiness	MILP, Tabu Search, Simulated Annealing
Yepes-Borrero, Villa, Perea, and Caballero-Villalobos (2020)	250	30						X	X				min makespan	MILP, GRASP
Bitar, Dauzère-Pérès, and Yugma (2021)	25	3						X	X				min makespan, resources - max number of jobs	Integer Programming
Raboudi, Aalpan, Mangione, Tissot, and Noel (2022)	7	4						X	X	X	X		min makespan	MILP
Hasani, Hosseini, and Sana (2022)	150	35						X					min makespan, costs	E-constrained MIP, NSGAII, SPEAII
Lei and He (2022)	350	30							X	X			min makespan	Adaptative Artificial Bee Colony
Wang, Wu, et al. (2022)	200	5						X					min processing progress, makespan	MIP, Benders Decomposition
Lei and Yang (2022)	200	20		X				X		X			min tardiness, makespan	Multi-Sub-Colony Artificial Bee Colony
de Alba, Nucamendi-Guillén, and Avalos-Rosales (2022)	400	20		X									min tardiness	MILP, Iterated Local Search
Zheng, Jin, Xu, and Liu (2022)	200	20									X		min makespan, costs	MILP, Heuristic, Improved Differential Evolution
Berthier et al. (2022)	126	49						X	X		X		min makespan	MILP, GA
Chen, Fathi, Khakifirooz, and Wu (2022)	90	30		X				X		X			min makespan - max number of jobs	MILP, Tabu Search
Wang, Wu, Chu, and Yu (2023)	200	3	X					X					max profit	MIP, Benders Decomposition
Wang, Song, bibetal (2023)	1200	20					X						max profit	MILP, Rolling Horizon Algorithm
Wang, Li, and Gong (2023)	100	6		X									min tardiness, makespan	Pareto-Based Memetic Algorith
Sanati et al. (2023)	100	20						X					min energy consumption	MILP, Fix & Relax Heuristic
Wang, Feng, and Wang (2023)	500	50						X					min makespan	MILP, Fly Optimization Algorithm
Rolim, Nagano, and Prata (2023)	200	6		X									min penalties	MILP, Adaptive Large Neighborhood Search
Assia, Jeffali, Barkany Abdellah, Ziani, and Bouchnaif (2023)	20	100								X		X	min makespan, energy consumption	MILP
Orts, Puertas, Ortega, and Garzón (2023)	1750	15						X					min makespan	Quadratic Unconstrained Binary Optimization
Santoro and Junqueira (2023)	72	9		X						X	X		min makespan	MILP
Muñoz-Díaz, Escudero-Santana, and Lorenzo-Espejo (2024)	300	10			X			X					min makespan	MILP, Simulated Annealing, Tabu Search
Chen et al. (2024)	225	15											min costs	Branch & Price, MILP
Fonseca, Figueiroa, and Toffolo (2024)	250	30						X					min makespan	Matheuristic
Yizhuo Zhu (2024)	350	6							X	X			min makespan, energy consumption	Dynamical Artificial Bee Colony
Elyasi, Selcuk, Özener, and Coban (2024)	200	20		X				X			X		min earliness, tardiness	MILP, Imperialist Competitive Algorithm
Lian, Zheng, Zhu, and Gao (2024)	81	10			X			X	X		X	X	min processing times, penalties - max matching degree	NSGAII, Variable Neighborhood Search
Kurt and Cetinkaya (2024)	200	20				X				X	X		min makespan	MILP, Heuristic

Ins: Instances, ST: Setup times, Res: Resources
A/R: Acceptance/Rejection criteria, D: Due/Release dates, P: Precedences, B: Batch processing, TW: Multiple Time Windows
Av: Machine Availability, El: Machine Eligibility, Sp: Machine Speed.

a concept they call machine calendar. This is a case where it is necessary to schedule parallel resources with different timetables, such as different weekly and daily shifts of workers or machines, non-working days, vacations, and preventive maintenance periods while considering the eligibility of resources. Kurt and Cetinkaya (2024) research proposes the same approach but treats these schedules as batches and proposes a simplified model. Both proposals will be described in more detail in Section 3.

Several proposed solution methods include exact approximations using, for example, bender decomposition algorithms (Wang et al.,

2023). In this work, a maximum threshold for completing the work and the possibility of accepting or rejecting them were addressed. On the other hand, the study of Chen et al. (2024) proposes a Branch and Price algorithm that minimizes the fixed and variable costs of using the machines. There are also approaches with MILP formulations such as Sanati et al. (2023) that sought to minimize the energy consumption in a UPM problem and compared its results with a heuristic that transformed this problem into a single-machine scheduling problem. Regarding approaches based on heuristic methods, Afzalirad and Rezaeian (2017) addresses the problem by comparing a multi-objective ant colony optimization algorithm with an NSGA-II, proving the superiority of the former. Lei and He (2022) proposed an artificial bee colony algorithm to solve the problem with resources and preventive maintenance. On the other hand, Wang, Li, and Gong (2023) described a Pareto-based memetic algorithm to minimize total tardiness and mentions that prior knowledge of neighborhood structures can ease and speed up the exploration of the search space. Approximations with other methods such as GRASP (Yepes-Borrero et al., 2020), iterative local search (de Alba et al., 2022), tabu search (Shahvari & Logendran, 2017b), and simulated annealing (Bektur & Saraç, 2019) are also found. The approach of Orts et al. (2023), where quantum annealing is used for the UPM with some degree of priority for the jobs, draws attention because of its scalability.

Although many reviewed papers work on the theoretical model of the UPM, this problem has diverse applications in various industries. For example, there are some such as the one by Wang, Song, bibetal (2023) that proposed two MILP formulations to maximize profits in a UPM with time windows applied to satellite observation scheduling. In their experiments, they were able to solve instances of up to 800 jobs and 200 machines. On the other hand, Lian et al. (2024) performs proactive scheduling with time uncertainty considering various machine speeds, machine eligibility, and resources. In their application case, they considered a multi-objective model that sought to minimize the average processing time, the penalty for fluctuation in oxygen consumption, and maximize the degree of match between machines for a steel plant. Additionally, a scheduling problem for oil and gas reservoir management considering release dates and resources is addressed in Avdeenko et al. (2017). This was solved through a mathematical formulation and an algorithm based on dynamic programming and approximate solutions.

Most of the reviewed researches of the last 9 years propose, for their respective variant, exact solution methods (mostly MILP formulations solved with commercial solvers) and compare them with a heuristic proposal where both trajectory-based and evolutionary methods are found. Among the mathematical formulations encountered, we searched for the characteristics of the defined variables and grouped them according to the relationship they measured in such a way that they were comparable with those presented in this paper. These are:

- Sequence relationship between each pair of jobs for a machine.
- Relationship of each job to its start or completion time.
- Job membership to a specific batch.

In addition, there are other common variables among approaches, such as the makespan and activation variables for disjunctive constraints. Most approaches focus on the relationship between each pair of jobs. However, as the number of jobs increases, the number of variables increases, and with it the computational burden. Therefore, it is worthwhile to compare the merits of each approach and to be aware of its limitations. In addition, to the best of our knowledge, there have been no approximations that seek to minimize the last shift, so we propose variations of the models found in such a way that this parameter is optimized.

In summary, Table 1 shows that the UPM is a scheduling problem that has been widely studied in recent years. Among the attributes of the problem most included in the various approaches are setup times (62% of the articles reviewed included it), deadline times for jobs (36%), machine eligibility (33%), and resource usage (31%). Other attributes have not been explored extensively in this problem such as job acceptance and rejection. This becomes relevant in certain industries where you have unstable production capacity and must choose the most profitable orders to meet demand and maximize profits (Wang et al., 2023). Another feature relates to multiple time windows where, in applications related to seasonal phenomena, measurements or jobs can only be made in specific time windows for each machine (Wang, Song, bibetal, 2023). On the other hand, while UPM considers that machines process the same order at different speeds, few studies consider the speed rate change within the same machine. This attribute becomes relevant when considering the machine wear and energy consumption (Assia et al., 2023), an objective function that, as shown in Table 1, has gained importance in recent years. This may be due to the growing interest in sustainability. Additionally, no approaches have been found considering preemption. This consists of interrupting a job in progress and continuing it later, which can improve the objective function depending on the number of jobs used (Fomin & Goldengorin, 2022). Regarding solution methods, it should be noted that there is a balance between exact, trajectory-based, and evolutionary algorithms. Likewise, there are approaches with reinforcement learning, (Perez Bernal, Salido, & March Moya, 2025; Wang et al., 2024). For example, Zampella et al. (2025) proposed a Multi-Agent Reinforcement Learning (MARL) approach for the UPM. Their study shows that while Single-Agent RL methods, such as Maskable PPO, perform well in small-scale scenarios, MARL offers scalability but struggles with coordination challenges. This opens the door to further exploration by incorporating new problem variants and refining learning strategies.

3. Mathematical formulations

In this section, we describe three mathematical models for the UPM with machine availability and eligibility. The first model, which we refer to as Model 1, measures the sequence relationship between each pair of jobs. Model 2 measures the allocation of jobs to a certain shift, calendar, or batch for a given machine, without specifying the internal order. Lastly, Model 3 considers the starting time of each job and machine assignment. First, we define the parameters and sets. Then, we describe the common and the specific variables for each model. Finally, we present the MILP formulations for makespan and last-shift minimization.

3.1. Parameters and sets

- N: number of jobs.
- Q: number of machines.
- $I = \{0, \dots, N\}$: set of jobs (0 is a dummy job).
- $M = \{1, \dots, Q\}$: set of machines.
- M_j: set of machines where job $j \in I$ can be performed.
- p_{jm}: duration of job $j \in I$ in machine $m \in M_j$.
- pm_j: minimum time of job $j \in I$ for all the machines $(\min\{p_{jm}\}_{m \in M_j})$.
- α: standard duration of a working shift.
- B_M: an appropriately large number that represents an upper bound for the makespan and it can be computed as the worst-case scenario resulting from the sum of the longest processing time for each job $j \in I$.
- S_m: set of shifts' end times for the machine $m \in M$. Shifts are obtained by rounding up the division of B_M by a predefined duration of a shift α.
- α_{sm}: set containing the length of the shift $s \in S_m$ for the machine $m \in M$ since the machines may have preventive maintenance that will shorten the operating time in that shift, otherwise the value is α.
- $T = \{1, \dots, B_M\}$: set of time intervals in which the schedule can take place.

3.2. Decision variables

All the proposed models share some common decision variables. For example, c_{max} measures the completion time of the last performed job which defines the makespan minimization objective. On the other hand, s_{max} corresponds to the completion time of the last shift used when performing a job for the last shift minimization approach. Despite that, each model considers different relationships between jobs, machines, or shifts.

- **Model 1**: for this proposal we include two types of variables. x_{ijms} takes the value of 1 if job i is performed just before job j in machine m and during the shift s, 0 otherwise. And variable c_j measures the completion time of job j.
- **Model 2**: this formulation includes two type of variables as well. Variable y_{jms} is 1 if job j is performed in machine m during the shift s, 0 otherwise. Variable z_{ms} is also a binary variable that lets us know, when it is equal to 1, if at least one job is performed on machine m during shift s, and 0 otherwise.
- **Model 3**: this approach considers the binary variable w_{jmt} that takes the value of 1 if job j is performed in machine m and begins at time interval t, 0 otherwise.

3.3. Makespan minimization

For the first objective function approach, we minimize the project makespan, which is measured by variable c_{max}. This objective is the same for the three models and it can be seen in Eq. (1).

$$\min \ Z = c_{max} \tag{1}$$

This objective function is one of the most common objectives used in literature as can be seen in Table 1.

3.4. First mathematical formulation: Model 1

We adapted the model of Rabadi, Moraga, and Al-Salem (2006) adding the required features to consider shifts. The constraints of Model 1 are presented by Eqs. (4) to (14).

This model looks alike and considers the same variables as the one proposed by Santoro and Junqueira (2023) but includes capacity constraints given how it was conceptualized. To simplify the model, we group the sum of the duration of the job if it is performed on the machine during a shift with the auxiliary variable $dura_{ij}$, which can be defined by Eqs. (2) or (3).

$$dura_{ij} = \sum_{m\in M}\sum_{s\in S_m} p_{jm}\cdot x_{ijms} \tag{2}$$

$$dura_{ij} = pm_j + \sum_{m\in M}\sum_{s\in S_m} \left(p_{jm} - pm_j\right)\cdot x_{ijms} \tag{3}$$

Note that Eqs. (2) and (3) are equivalent, but the latter helps to strengthen Constraints (6) from Model 1.

In this formulation, Constraint (4) ensures that for each job, excluding the dummy, there must be only one machine to process it during a specific shift. Constraint (5) states that each job must have a predecessor and a successor on the same machine (the dummy job would be the start and end of the scheduling on each machine). Constraint (6) controls that the completion time of a job is greater than its processing time plus the completion time of the previous job. In addition, the fixed value of the minimum duration of the job on any machine where it can be executed, pm_j, is always added to shorten the search space. Constraints (7) and (8) establish that the dummy is the starting job on all machines, with starting time equal to zero. Constraint (9) defines the makespan as a time greater or equal to the completion time of any job (including the last one). Constraints (10) and (11) control jobs to be performed only within specific shifts and not outside of machine working calendars. Constraint (12) states that the size of the shift limits the total length of jobs that can be performed within it. Finally, Constraints (13) and (14) establish the domain of the decision variables.

3.5. Second mathematical formulation: Model 2

This is the simplest model as it does not measure the order of the jobs within each shift. It is similar to the model used in Kurt and Cetinkaya (2024) but adds one set of constraints to limit the interaction between the variables y_{ims} and z_{ms}. The constraints of this formulation are presented below by Eqs. (15) to (20).

In this formulation, as described before, the jobs are restricted to be performed in one of the available machines and within a specific shift (Constraint (15)). Constraint (16) limits the total length of jobs that can be performed during a shift depending on their processing times. Constraint (17) defines the duration of the makespan as the sum of the durations of the jobs of the last interval used and its corresponding starting time. Constraint (18) enforces that shifts are only used when jobs are executed within them. Finally, Constraints (19) and (20) establish the domain of the decision variables.

3.6. Third mathematical formulation: Model 3

This model is inspired by the time constraints in Christofides, Alvarez-Valdes, and Tamarit (1987) for project scheduling with resource constraints. However, our approach is novel, as we develop a new model specifically tailored for the UPM, incorporating eligibility constraints and machine availability while adapting key elements to better suit this context. For this purpose, we included the sub-indexes related to the use of machines and the control of the execution of jobs in determined time intervals within shifts (Eqs. (21) to (26)).

$$\sum_{\substack{i\in I\\ i\neq j}}\sum_{m\in M_j}\sum_{s\in S_m} x_{ijms} = 1 \qquad \forall\ j\in I,\ j>0 \tag{4}$$

$$\sum_{\substack{i\in I\\ i\neq h}}\sum_{s\in S_m} x_{ihms} - \sum_{\substack{j\in I\\ j\neq h}}\sum_{s\in S_m} x_{hjms} = 0 \qquad \forall\ m\in M,\ h\in I,\ h>0 \tag{5}$$

$$c_j \geq c_i + dura_{ij} - B_M\cdot\left(1 - \sum_{m\in M}\sum_{s\in S_m} x_{ijms}\right) \qquad \forall\ i\in I,\ j\in I,\ j>0 \tag{6}$$

$$\sum_{j\in I}\sum_{s\in S_m} x_{0jms} = 1 \qquad \forall\ m\in M \tag{7}$$

$$c_0 = 0 \tag{8}$$

$$c_{max} \geq c_j \qquad \forall\ j\in I,\ j>0 \tag{9}$$

$$c_j \geq s - \alpha_{sm} + p_{jm}\cdot\sum_{\substack{i\in I\\ i\neq j}} x_{ijms} - B_M\cdot\left(1 - \sum_{\substack{i\in I\\ i\neq j}} x_{ijms}\right) \qquad \forall\ m\in M,\ s\in S_m,\ j\in I,\ j>0 \tag{10}$$

$$c_j \leq s + \left(B_M - s\right)\cdot\left(1 - \sum_{\substack{i\in I\\ i\neq j}} x_{ijms}\right) \qquad \forall\ m\in M,\ s\in S_m,\ j\in I,\ j>0 \tag{11}$$

$$\sum_{i\in I}\sum_{\substack{j\in I\\ i\neq j}} p_{jm}\cdot x_{ijms} \leq \alpha_{sm} \qquad \forall\ m\in M,\ s\in S_m \tag{12}$$

$$x_{ijms}\in\{0,1\} \qquad \forall\ i\in I,\ j\in I,\ m\in M,\ s\in S_m \tag{13}$$

$$c_j \geq 0 \qquad \forall\ j\in I \tag{14}$$

$$\sum_{m\in M_i}\sum_{s\in S_m} y_{ims} = 1 \qquad \forall\ i\in I,\ i>0 \tag{15}$$

$$\sum_{i\in I} p_{im}\cdot y_{ims} \leq \alpha_{ms} \qquad \forall\ m\in M,\ s\in S_m \tag{16}$$

$$c_{max} \geq \left(s - \alpha_{ms}\right)\cdot z_{ms} + \sum_{i\in I} p_{im}\cdot y_{ims} \qquad \forall\ m\in M,\ s\in S_m \tag{17}$$

$$z_{ms} \geq y_{ims} \qquad \forall\ m\in M,\ s\in S_m,\ i\in I \tag{18}$$

$$y_{ims}\in\{0,1\} \qquad \forall\ i\in I,\ m\in M,\ s\in S_m \tag{19}$$

$$z_{ms} \in \{0,1\} \qquad \forall\ m \in M,\ s \in S_m \tag{20}$$

$$\sum_{m \in M_i} \sum_{s \in S_m} \sum_{t=s-\alpha_{ms}+1}^{s} w_{imt} = 1 \qquad \forall\ i \in I,\ i > 0 \tag{21}$$

$$\sum_{t=s-\alpha_{ms}+1}^{s} (t \cdot w_{imt}) + (p_{im} - 1) \cdot w_{imt} \le s \qquad \forall\ m \in M,\ s \in S_m,\ i \in I,\ i > 0 \tag{22}$$

$$\sum_{t=s-\alpha_{ms}+1}^{s} t \cdot w_{imt} \ge s - \alpha_{ms} - B_M \cdot \left(1 - \sum_{t=s-\alpha_{ms}+1}^{s} w_{imt}\right) \qquad \forall\ m \in M,\ s \in S_m,\ i \in I,\ i > 0 \tag{23}$$

$$c_{max} \ge \sum_{t \in T} \sum_{m \in M} \left(t \cdot w_{imt} + (p_{im} - 1) \cdot w_{imt}\right) \qquad \forall\ i \in I,\ i > 0 \tag{24}$$

$$\sum_{\substack{i \in I \\ i>0}} \sum_{\substack{h=0 \\ t-h \neq 0}}^{min\{p_{im},\ t\}-1} w_{im(t-h)} \le 1 \qquad \forall\ t \in T,\ m \in M \tag{25}$$

$$w_{imt} \in \{0,1\} \qquad \forall\ i \in I,\ t \in T,\ m \in M \tag{26}$$

Constraint (21) ensures that each job can only start within one of the periods that elapse between shifts and on a single machine. Constraints (22) and (23) guarantee that if jobs are performed during shifts, their execution does not overlap with other shifts. Constraint (24) defines the makespan as the duration of the last job plus its starting time. Constraint (25) ensures that no job can start while another job is in progress in the same machine. Finally, Constraint (26) presents the domain of the decision variables.

3.7. Last shift minimization

In an alternative way, we intend to minimize the completion time of the last used shift. This objective function can be represented by Eq. (27), by using the decision variable s_{max}. This objective function can be used in all proposed models as a replacement for makespan minimization.

$$\min\ Z = s_{max} \tag{27}$$

Regarding this scenario, the formulations presented remain identical except for the makespan constraint, which is replaced by the Constraints (28) to (30). These are interpreted similarly but relate to the end time of the last shift in which a job is performed. Regarding Model 1, Constraint (9) is replaced by Constraint (28). For Model 2, Constraint (29) replaces (17). Finally, for Model 3, Constraint (30) replaces (24).

$$s_{max} \ge \sum_{i \in I} \sum_{m \in M} \sum_{s \in S_m} (s \cdot x_{ijms}) \qquad \forall\ j \in I,\ j > 0 \tag{28}$$

$$s_{max} \ge s \cdot z_{ms} \qquad \forall\ m \in M,\ s \in S_m \tag{29}$$

$$s_{max} \ge \sum_{t=s-\alpha_{ms}+1}^{s} (s \cdot w_{imt}) \qquad \forall\ m \in M,\ s \in S_m,\ i \in I,\ i > 0 \tag{30}$$

In this approach, the s_{max} corresponds to the completion time of the shift in which the last job is performed. In that sense, the optimal makespan and the last shift are very related, but the second one does not take into account the sequencing of the jobs along the shifts as this objective function remains unchanged to different permutations of jobs.

Note that, given an optimal s_{max}, i.e. s^*_{max}, and an optimal c_{max}, i.e. c^*_{max}, the starting time of the shift s^*_{max} plus the minimum job processing time is a lower bound for the optimal makespan, c^*_{max}.

Table 2
Processing times and machines where jobs can be performed.

	I_1	I_2	I_3	I_4	I_5	I_6	I_7	I_8	I_9	I_{10}
M_1	2	5	5	2			5		3	3
M_2		5	3	5	2	3	5	3	3	2
M_3	5		4	4	2		4	3		

Table 3
Available shift time per machine where jobs can be scheduled.

Shift	S_1	S_2	S_3	S_4	S_5
M_1	5	0	5	5	5
M_2	3	5	5	5	5
M_3	5	5	5	5	5

Similarly, the finish time of the shift s^*_{max} is an upper bound for the optimal makespan, c^*_{max}. That is:

$$s^*_{max} - \alpha + \min\{pm_j\} \le c^*_{max} \le s^*_{max}$$

That means that the absolute error (AE), given an optimal s_{max}, only depends on the standard length of the shifts and the minimum job processing time, while relative error (GAP) reduces with the number of required shifts that can be approximated as $ns = \frac{s^*_{max}}{\alpha}$.

$$\text{AE} \le s^*_{max} - \left(s^*_{max} - \alpha + \min\{pm_j\}\right)$$

$$\text{AE} \le \alpha - \min\{pm_j\}$$

$$\text{GAP} \le \frac{AE}{s^*_{max}} = \frac{\alpha - \min\{pm_j\}}{s^*_{max}} \le \frac{\alpha}{s^*_{max}} = \frac{1}{ns}$$

This approach aims to speed up the convergence to optimal solutions, as can be deduced from Tables 5 and 6, and is a useful tool for companies whose operations are governed by machines or working calendars.

3.8. Practical example

The following example intends to help the reader to understand the problem, the parameters considered, and the expected result. For this purpose, 10 jobs with durations between 2 and 5 intervals, 3 machines, and shifts of 5 time intervals (α parameter) are considered. Table 2 presents the processing times for each job on each machine. In addition, this table has the machine eligibility information showing a blank space in the machines where a specific job cannot be performed. This matrix relates to the p_{jm} parameter. From this, the value of pm_j could be extracted for each activity being the smallest value in each column (2 for the first job, 5 for the second job, and so on). Finally, M_j contains the indices of the machines that can process each activity. For the first activity, it would be a set with machines 1 and 3, 1 and 2 for the second, the third job can be performed on all machines, and so on.

On the other hand, Table 3 contains the machine availability information (for the 5 time interval shifts). As mentioned before, the machine may not be fully operational due to scheduled outages, preventive maintenance, or unexpected failures. For example, machine 1 is not available at all during shift 2, and machine 2 only has 3 time slot availabilities during the first shift. The maintenance time within a given time slot is defined in such a way that the remaining available time in the shift remains continuous. To ensure this continuity, the model assumes that the unavailable time is placed at the beginning of the shift. However, once this assumption is established, the exact placement — whether at the beginning or the end — does not impact the overall scheduling. This consideration does not affect the last shift minimization objective, although it could influence the makespan. Depending on the time-bound considered in the planning, there may be a greater number of shifts. This table represents the information contained in the α_{sm} parameter. In this sense, the parameter S_m would have a similar

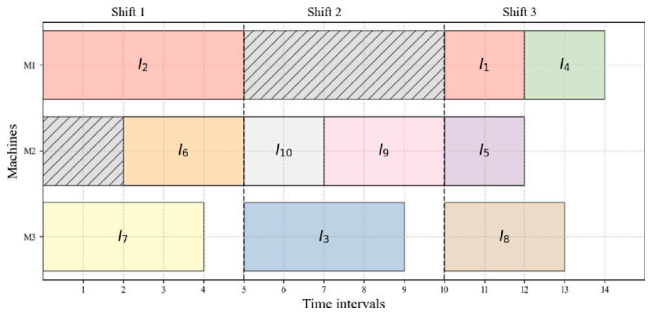


Fig. 1. Gantt chart of the optimal solution for the test data set.

structure but considering the shift's end time (i.e. $[5, 10, 15, \ldots]$) for each machine.

Fig. 1 shows a Gantt chart representing the solution for the stated problem. Each scheduled job is placed in an appropriate time frame while respecting the machine shifts. For example, job 3 cannot start in time slot 4 because it would be disrupted by the beginning of the second shift. On the other hand, the first machine has no job assigned for this shift nor does the second machine for the first two intervals of the first shift due to the above-mentioned machine availability constraints. In this solution, the makespan would be 14 and the finishing time of the last shift would be 15 time intervals. This is a detailed solution provided by both Models 1 and 3. In the case of Model 2, we would know which jobs are assigned to each shift, however, the start and end time of each job would be unknown.

3.9. Computational complexity

To understand the complexity of the model, we consider the number of variables and constraints, as these elements significantly influence the difficulty of finding an optimal solution. A higher number of variables increases the dimensionality of the search space, while additional constraints can tighten the feasible region, making it easier or more challenging to explore. Although complexity is not solely determined by these factors, analyzing them provides a useful initial perspective on the scalability and computational effort required to solve the problem.

For Model 1, the number of binary decision variables grows proportionally to $O(N^2QS)$ (where $S = \max_{m \in M} |S_m|$) due to the sequencing variables x_{ijms}. The number of constraints follows a $O(max(N^2, NQS))$ complexity, leading to scalability issues as the number of jobs increases. In Model 2, the number of binary variables is $O(NQS)$, and constraints are also linear with respect to these parameters. This results in better computational efficiency, as shown in the experimental results of Section 4. Model 3 introduces time-dependent variables, increasing complexity to $O(NQ|T|)$, where T represents the time interval array which cardinality is α times larger than the S present in its counterparts. This model allows finer granularity in scheduling, and the number of constraints increases proportionally.

Table 4 presents the formulas used to estimate the number of variables (both continuous and integer) and constraints for each model. These formulas allow us to quantify the problem size as a function of its parameters and understand how complexity scales with problem size. The data shown correspond to the makespan minimization problem. For the last shift minimization, the same equations are applied, except for the number of constraints in Model 3, where the formula changes to $N(3QS + 1) + TQ$. It is important to note that in some equations a unit is added to the N parameter given the dummy activity considered in the models.

Table 4
Formula to estimate the number of variables and constraints per model for makespan minimization.

Md	Vars		Constrains
	Cont	Int	
1	$N+2$	$QS(N+1)^2$	$1 + N(3 + Q + N + 2SQ) + Q(1 + S)$
2	1	$QS(N+2)$	$N + Q(3S + SN)$
3	1	$Q\|T\|(N+1)$	$\|T\|Q + 2N(1 + QS)$

4. Computational results

In this section, we describe the performed experiments and delve into the results obtained for each objective function and then an analysis with larger instances.

4.1. Experimental setup

To the best of our knowledge, there are no instances available for UPM with eligibility and machine availability. Most authors create their instances using common parameters from literature to test their models. To establish a benchmark and to be able to further test models and compare them, we stored the generated instances in an open repository. Moreover, since the purpose of this paper is to highlight the tradeoffs of each model rather than generating bigger size problems, we focus on generating instances that allow us to visualize how these parameters influence the performance of the models. The factors we decided to alter are the number of jobs (10 and 25), number of machines (3 and 6), shift size (10 and 20-time intervals), how many machines can process the jobs (sparse and dense eligibility matrix), jobs processing times (mixed jobs between 1 and 10-time intervals and long jobs between 5 and 10), and for ensuring repeatability, 3 seeds for each setup are considered. The process for generating the instance is described below.

We generate the job processing times for each configuration and seed for each machine using a discrete uniform distribution between the given intervals. Then, we estimate a large number with the sum of the longest time for each job to establish the planning horizon. With this planning horizon, we assign shifts according to their predefined length. Additionally, with a probability of 0.5, we modify a random machine shift to include maintenance or unavailability (this time corresponds to a random number between 1 and the predefined length of the shift). This means that, for each instance, we are considering working calendars and maintenance periods. For the eligibility criteria, we consider "sparse" as selecting the machine to process the job with a probability of 0.3 resulting from a binomial distribution. For the "dense", the probability used is 0.7. In the case no machine can process a job, we select a random machine and fix it to make the instance feasible. The result of varying these parameters comprises 96 instances.

To compare the three models with the proposed objectives and functions proposed, we needed 576 runs. We solved it using Gurobi Optimizer version 11.0.1 and on a computer with Intel(R) Xeon(R) processor with E5-2670 v3 @ 2.30 GHz 16-core CPU, 377 GB DIMM DDR4 memory, and 1TB HDD storage. A time limit of 1 h was set for Gurobi. If the optimal solution is not achieved, the best feasible solution found is reported.

4.2. Makespan minimization analysis

It is noteworthy that Models 2 and 3 reach the optimum in all instances with the maximum computation time set at one hour. On the contrary, Model 1 exhibits higher computational demand for reaching a GAP value lower than 0.5%. It could find the optimal value in 69 instances out of 96. Additionally, this approach did not find feasible solutions in 7 of the instances. By analyzing the average times of each model, the behavior and scalability of these models are revealed. Particularly the average times were 1315,09, 0,99, and 1,96 s for

Table 5
Model performance under parameter variation for makespan minimization.

Md	Ind	Jobs		Machines		Shift length		Eligibility		Duration	
		10	25	3	6	10	20	dense	sparse	1-10	5-10
1	Gap %	0,00%	16,56%	10,53%	4,92%	6,23%	8,99%	11,39%	3,95%	8,80%	6,48%
	Time s	28,26	2601,92	1513,32	1116,86	1059,96	1570,22	1491,29	1138,89	1326,29	1303,89
	Vars #	3899,13	52586,00	17806,63	38678,50	37389,63	19095,50	28242,56	28242,56	27498,50	28986,63
	Cons #	855,13	4783,25	1933,63	3704,75	3569,38	2069,00	2819,19	2819,19	2756,38	2882,00
2	Gap %	0,00%	0,00%	0,00%	0,00%	0,00%	0,00%	0,00%	0,00%	0,00%	0,00%
	Time s	0,03	1,97	1,11	0,88	1,27	0,72	1,34	0,66	0,74	1,26
	Vars #	386,50	2100,25	784,00	1702,75	1645,00	841,75	1243,38	1243,38	1209,63	1277,13
	Cons #	427,63	2202,00	835,13	1794,50	1734,13	895,50	1314,81	1314,81	1279,50	1350,13
3	Gap %	0,00%	0,00%	0,00%	0,00%	0,00%	0,00%	0,00%	0,00%	0,00%	0,00%
	Time s	0,13	3,79	2,35	1,58	1,01	2,91	2,78	1,14	2,01	1,92
	Vars #	4744,75	27041,00	10014,75	21771,00	15714,75	16071,00	15892,88	15892,88	15459,75	16326,00
	Cons #	1093,75	4977,50	1926,25	4145,00	3758,75	2312,50	3035,63	3035,63	2953,75	3117,50

Table 6
Model performance under parameter variation for last shift minimization.

Md	Ind	Jobs		Machines		Shift length		Eligibility		Duration	
		10	25	3	6	10	20	dense	sparse	1-10	5-10
1	Gap %	0,00%	1,38%	1,35%	0,00%	1,38%	0,00%	0,00%	1,32%	0,00%	1,26%
	Time s	0,57	669,06	525,33	144,30	570,37	99,26	275,85	393,78	405,98	263,65
2	Gap %	0,00%	0,00%	0,00%	0,00%	0,00%	0,00%	0,00%	0,00%	0,00%	0,00%
	Time s	0,02	0,42	0,28	0,16	0,25	0,19	0,30	0,14	0,17	0,27
3	Gap %	0,00%	0,00%	0,00%	0,00%	0,00%	0,00%	0,00%	0,00%	0,00%	0,00%
	Time s	0,22	1,45	1,07	0,60	0,66	1,02	1,12	0,56	1,07	0,60

Models 1, 2, and 3, respectively. Table 5 reports the performance results for the makespan minimization of the proposed models. The metrics used are the GAP percentage, computation time (in seconds), and the number of variables and constraints. Each value corresponds to the mean value of the metrics for the instances when fixing a specific parameter. The factors considered are the number of jobs, machines, shift size, eligibility matrix features, and intervals used for job duration. For example, all solved instances containing 10 jobs (48 instances) are averaged and compared to those containing 25 jobs (48 remaining instances). The average aggregates the variation in the other parameters for the three seeds considered. The same process is repeated for the other factors.

These results demonstrate that the most influential factor for model performance is the number of jobs. An increase from 10 to 25 jobs causes Model 1 to start finding issues with finding even feasible solutions. This is explained by the fact that the variables proposed depend on the relationship between each pair of jobs, implying that the average increase in the number of variables is 1349% (in contrast to Models 2 and 3, whose variables depend on a single job with a 557% of increase on average). A similar increase is found for the constraints since most of these are associated with each job. On the other hand, Model 2 presents lower computation times than Model 3, being 48% faster.

Regarding the increase in the number of machines, despite the larger problem size, it makes it easier for the model to find the optimum. This is mainly explained by the fact that, as there are more machines for the same set of jobs, the distribution of jobs according to the machine eligibility parameter makes the number of feasible permutations to be evaluated smaller. This reduces the processing times by an average of 27%. When having a shift of longer duration, it is expected that in the same time horizon, there will be fewer shifts, thus reducing the constraints and variables (except for Model 3 where the variable does not include a subindex for the shift). However, the models' improvement by including shifts with different lengths is not clear-cut. Nevertheless, when a sparse eligibility matrix is used, the computation time of the models is improved by 44% since it narrows the possible structures to evaluate, thus simplifying the problem. On the other hand, considering jobs with longer durations (between 5 and 10 time intervals) does not significantly influence model performance. The small variations in constraints and variables are due to the pre-calculations of time horizons and the number of shifts that depend on the total duration of the jobs.

4.3. Last shift minimization analysis

Table 6 shows the results for the objective function of minimizing the last shift in the same structure as above (rows containing the metrics for each model and columns with the levels for each parameter). However, the number of variables and constraints are not taken into account since there is no significant variation. The results are consistent with the previous analysis with a significant increase in computational demand as the number of jobs increases. The models exhibit a decrease in the computational times with an increasing number of machines as well. On the other hand, shift length is a parameter that is not conclusive in terms of algorithm performance, since its behavior depends on the combination of the model and objective function that is defined. The same effect occurs when varying the distribution of the duration of jobs. Unlike the minimization of the makespan, in this case, there is a greater percentage difference in the computation times after changing this parameter, however, it affects the models differently. Specifically, Models 1 and 3 improve their performance by an average of 39% after changing the average job duration (to 5–10 time intervals). Interestingly, the performance of the models in terms of computational time improves considerably by changing the objective function. Model 1 achieved the optimum in less time than the established limit in 91 of the instances. However, in one instance it reached a GAP of more than 0.5% and in 4 instances it did not find feasible solutions. The particularity of these instances where it failed to reach the optimum is those are configurations that we will refer to from now on as "difficult", i.e., an instance with a higher number of jobs, a lower number of machines (25 and 3 respectively) and a dense eligibility matrix. As in the previous experimentation, both Models 2 and 3 achieved the optimum for all instances. The average computation times were 334,81, 0,22, and 0,83 for Models 1, 2, and 3, respectively. This represents an average improvement of 70% concerning makespan minimization.

Table 7
Models' performance with increasing number of jobs.

Md	Objective	Ind	50	100	200	400	800	1600
2	Makespan	Gap %	0,00%	0,00%	0,00%	24,04%	81,11%	100,00%
		Time s	13,62	52,04	682,92	3174,50	3600,00	3600,00
	Last shift	Gap %	0,00%	0,00%	0,00%	0,00%	40,85%	100,00%
		Time s	2,36	14,26	281,78	1278,59	3564,28	3600,00
3	Makespan	Gap %	0,00%	0,00%	0,00%	0,00%	100,00%	100,00%
		Time s	36,76	203,80	534,66	2183,79	3600,00	3600,00
	Last shift	Gap %	0,00%	0,00%	0,00%	0,00%	98,38%	-
		Time s	8,53	63,48	624,95	1841,68	3600,00	-

4.4. Larger instances experimentation

The last experiment could not verify the scalability of Models 2 and 3. Therefore, we selected the "difficult" setup: a shift of 10 time slots, we increased the number of jobs to 50, 100, 200, 400, 800, and 1600. Three instances for each experiment were created by varying the seed, using the same methodology described in Section 4.1.

From 50 and 100 jobs onwards, the first model failed to find feasible solutions in the time limit provided. Therefore, having found its threshold, Model 1 is omitted from the following analysis. Table 7 summarizes the results of the experiments. The rows display the average results for the percentage GAP and computing time in seconds for each objective function used in Models 2 and 3. The columns present the number of jobs under consideration, ranging from 50 to 1600.

In the case of 1600 jobs, the process was interrupted due to a lack of memory, and therefore, only the result of the first seed was reported. The models reached their limits for this number of jobs. Alternatively, the optimum was still reached in reasonable times with 200 jobs. Similarly, the benefit of the last shift minimization function, which exhibits better times and average GAPs, was reported (without accounting for the 1600 jobs of Model 3 where it could not find feasible solutions). An interesting aspect is how the gap between the computing times of the models closes as the number of jobs increases. With a low number of jobs (less than 100), Model 2 outperforms its counterpart in terms of computation time being 72% faster, however, when this parameter increases, Model 3 gradually becomes more relevant. This result is highly dependent on the time boundary defined for the problem since, by considering variables with a time-dependent subindex, the problem size increases as this and job durations change. Regardless, by addressing the order of the jobs, this model is a reliable representation for each job within intervals allowing the scaling and addition of a larger number of features such as machine and sequence-dependent setup times. The latter parameter cannot be included in Model 2 since its conceptualization is based on a lack of knowledge of the internal job order.

Figs. 2(a) and 2(b) summarize the performance in the processing time of the three models compared for both makespan and last shift minimization, respectively. On the other hand, Fig. 2(c), presents the evolution of the number of variables as the number of jobs increases. In these figures, we can see that Model 1 reaches the established computational threshold of 1 h first and is no longer used for comparison. For this number of jobs, the computation times of Models 2 and 3 are lower as well as the number of variables. When increasing these quantities, it is seen that both models compete, where Model 3 shows a better performance in the makespan minimization and Model 2 in the last shift objective function. The breach gradually narrows due to the established time limit (as well as to the reported GAP). One tends to assume that, as the maximum computation time increases, the differences between both models decrease, however, when the number of variables is analyzed, it is seen that the slope of growth of the variables is greater for Model 3, becoming a bigger problem to solve. This growth is attributable to a greater number of jobs, which increases the number of shifts required to solve the problem and therefore the number of time intervals. Each job, on each machine and in each time interval has an associated variable, unlike Model 2 where each job has a machine and an associated shift. For the scenario assessed, Model 3 has approximately 10 times more variables than Model 2 (which corresponds to the duration of the proposed shift).

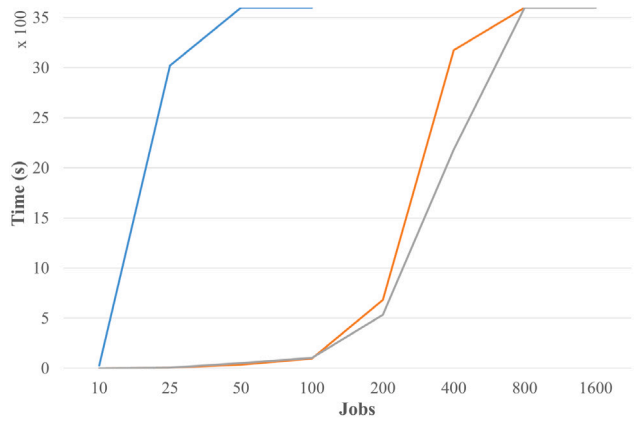
Table 8
Shift length and eligibility experiments with 400 jobs.

Md	Objective	Ind	Shift length		Eligibility	
			10	20	dense	sparse
2	Makespan	Gap %	12,57%	0,00%	12,57%	0,00%
		Time s	2840,11	1077,21	2475,05	1442,28
	Last shift	Gap %	0,00%	0,00%	0,00%	0,00%
		Time s	1323,39	1086,26	1556,76	852,88
3	Makespan	Gap %	0,00%	24,98%	16,56%	8,43%
		Time s	2289,39	3514,77	2996,80	2807,36
	Last shift	Gap %	0,00%	96,73%	48,25%	48,48%
		Time s	1614,12	3600,00	2725,00	2489,76

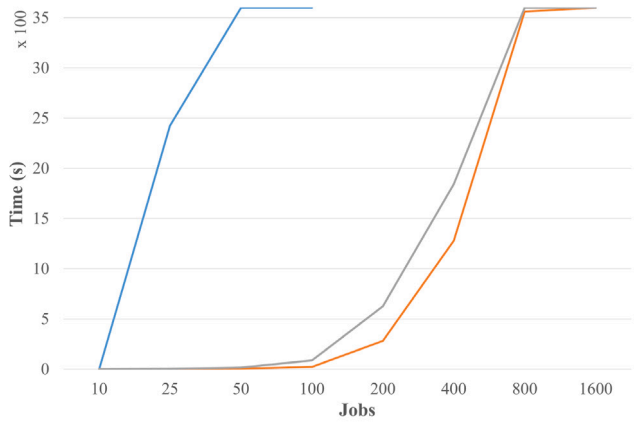
4.5. Advanced instance complexity analysis

As shown in Table 7, from 400 jobs onwards, significant variations in computation times and average GAP were found in both models. Therefore, to understand the real impact of parameters such as shift length and machine eligibility in Models 2 and 3, additional experimentation was performed. For this, 400 jobs (with a duration between 1 and 10 time slots), and 3 machines were selected. The shift size (10 and 20-time slots) and the machine eligibility parameter (sparse and dense) were varied. This resulted in 12 instances including the 3 seeds used. The results are presented in Table 8. Each result shows the average of GAP and time for the instances grouped according to the level of the shift length and machine eligibility matrix feature. This is reported for each objective function and model.

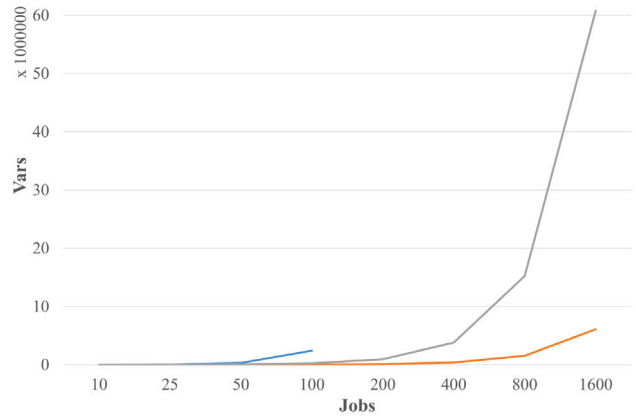
It was found that shift size is a factor that impacts the performance of both models in the opposite way. Model 2, by not considering the internal order of the jobs, does not consider the complete range of possible permutations (since it only counts the total sum of job durations). Therefore, increasing the shift lengths reduces the number of variables and decreases its computation times. Thus, this model benefits from having fewer shifts of longer duration. On the other hand, Model 3 increased its average GAP and computation times after considering longer shifts. This happens because more possible combinations are evaluated, thus increasing the problem size. Regarding the eligibility settings, we confirmed that a dense matrix makes both models more difficult to solve since it increases the number of possible combinations to evaluate for each machine. This can be seen in both computation times and average GAP. Model 3, minimizing the last shift, seems to present a similar GAP, but this behavior is because both factors absorb the internal impact of the increase in shift duration, the most significant parameter for this setup.



(a) Models computation times for makespan minimization.



(b) Models computation times for last shift minimization.



(c) Number of variables increases for each model.

Fig. 2. Performance of Model 1 (blue lines), Model 2 (orange lines), and Model 3 (gray lines).

5. Conclusions

In this research, we compared two models adapted from the literature to solve the UPM scheduling problem with machine eligibility and availability. Additionally, we propose a third model that aims to achieve a trade-off between both approaches. The models have different variable structures and can capture different relationships between parameters. Experimentation was performed using different factors to understand the performance and capability of the models when minimizing the makespan. Additionally, a new objective not previously explored in the literature for this problem is proposed: the minimization of the last shift. Model 1, which is based on the relationship between each pair of jobs and their assignment to a shift and machine, exhibits lower performance as the number of jobs increases and fails to find feasible solutions for more than 50 jobs. On the other hand, Model 2, which considers the allocation of each job to a shift and a machine, has the best computation times, reaching the optimum in all instances up to 200 jobs. However, this model lacks detail in the internal sequencing between shifts, therefore, with longer jobs and working shifts, the solution can be complemented by any heuristic rule since any order gives an equivalent solution for each shift. Model 3 considers the relationship of the starting time interval for each job on a specific machine. This approach reaches the optimum with up to 400 jobs in less than the specified time limit. Although its computation times are longer than Model 2, this proposal is highly scalable since, by considering job order, it supports the addition of further parameters like sequence and machine-dependent set-up times, operators' skills, job partitioning, and resource consumption. However, this model depends on a pre-calculation of a time horizon. Therefore, as the number of jobs continues growing, variables increase with a greater slope due to the increased time required to complete the schedule. If estimated heuristically, the performance of the model can be improved. On the other hand, the reduction in computational time when considering the last shift minimization objective function is noteworthy. This provides the models with the flexibility to move jobs between shifts easier.

The findings of this research provide a valuable framework for decision-making in the selection of scheduling models in demanding industrial environments. In particular, the ability to effectively model attributes like machine availability and eligibility, along with the optimization of metrics including makespan and completion time of the last shift, allows companies to balance operational efficiency with computational limitations. In high-demand environments where planned outages and efficient shift scheduling are critical, minimizing the last shift stands out as a strategic objective. This approach not only reduces operating costs but also improves planning and resource allocation, maximizing system performance. The flexibility of the proposed models to adapt to various parameters and configurations highlights their applicability in modern industry, which faces challenges such as cost reduction and continuous process improvement.

Future work includes extending the formulation of Model 3 to include resources of various types (renewable, non-renewable, and storage), operators' skills, or job sequence-dependent setup times. In addition, a metaheuristic approach can be proposed to provide efficient solutions in less time. Moreover, it is proposed to extend the design of experiments and consider more levels for the factors to assess the detailed evolution of the performance of the proposals. Likewise, we plan to perform experiments by including and removing attributes to further characterize the problem.

CRediT authorship contribution statement

Alejandro Uribe: Conceptualization, Methodology, Software, Validation, Investigation, Resources, Writing – original draft, Writing – review & editing. **Urtzi Otamendi:** Conceptualization, Methodology, Software, Validation, Investigation, Resources, Writing – original draft, Writing – review & editing. **Arkaitz Artetxe:** Conceptualization, Methodology, Validation, Investigation, Resources, Writing – original draft, Writing – review & editing, Supervision. **Juan Carlos Rivera:** Conceptualization, Methodology, Validation, Investigation, Writing – original draft, Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

Abed, M., & Mohmad Kahar, M. N. (2023). Review on unrelated parallel machine scheduling problem with additional resources. *Iraqi Journal for Computer Science and Mathematics*, *4*, 224–236. http://dx.doi.org/10.52866/ijcsm.2023.02.02.020.

Afzalirad, M., & Rezaeian, J. (2016). Resource-constrained unrelated parallel machine scheduling problem with sequence dependent setup times, precedence constraints and machine eligibility restrictions. *Computers & Industrial Engineering*, *98*, 40–52. http://dx.doi.org/10.1016/j.cie.2016.05.020, URL https://www.sciencedirect.com/science/article/pii/S0360835216301619.

Afzalirad, M., & Rezaeian, J. (2017). A realistic variant of bi-objective unrelated parallel machine scheduling problem: NSGA-II and MOACO approaches. *Applied Soft Computing*, *50*, 109–123. http://dx.doi.org/10.1016/j.asoc.2016.10.039, URL https://www.sciencedirect.com/science/article/pii/S1568494616305634.

Aggoune, R. (2004). Two-job shop scheduling problems with availability constraints.. In *Proceedings of the 14th international conference on automated planning and scheduling* (pp. 253–259).

Assia, S., Jeffali, F., Barkany Abdellah, E., Ziani, E., & Bouchnaif, J. (2023). Bi-objective unrelated parallel machine joint scheduling of jobs and preventive maintenance with a dynamic speed-scaling technique. *Materials Today: Proceedings*, *72*, 3454–3462. http://dx.doi.org/10.1016/j.matpr.2022.08.103, Fifth edition of the International Conference on Materials & Environmental Science (ICMES 2022) URL https://www.sciencedirect.com/science/article/pii/S2214785322052592.

Avdeenko, T. V., Mezentsev, Y. A., & Estraykh, I. V. (2017). Heuristic approach to unrelated parallel machines scheduling under availability and resource constraints. *IFAC-PapersOnLine*, *50*(1), 13096–13101. http://dx.doi.org/10.1016/j.ifacol.2017.08.1998, 20th IFAC World Congress URL https://www.sciencedirect.com/science/article/pii/S2405896317326368.

Barck-Holst, P., Nilsonne, Å., Åkerstedt, T., & Hellgren, C. (2017). Reduced working hours and stress in the Swedish social services: A longitudinal study. *International Social Work*, *60*(4), 897–913. http://dx.doi.org/10.1177/0020872815580045.

Bektur, G., & Saraç, T. (2019). A mathematical model and heuristic algorithms for an unrelated parallel machine scheduling problem with sequence-dependent setup times, machine eligibility restrictions and a common server. *Computers & Operations Research*, *103*, 46–63. http://dx.doi.org/10.1016/j.cor.2018.10.010, URL https://www.sciencedirect.com/science/article/pii/S0305054818302697.

Berthier, A., Yalaoui, A., Chehade, H., Yalaoui, F., Amodeo, L., & Bouillot, C. (2022). Unrelated parallel machines scheduling with dependent setup times in textile industry. *Computers & Industrial Engineering*, *174*, Article 108736. http://dx.doi.org/10.1016/j.cie.2022.108736, URL https://www.sciencedirect.com/science/article/pii/S0360835222007240.

Bitar, A., Dauzère-Pérès, S., & Yugma, C. (2021). Unrelated parallel machine scheduling with new criteria: Complexity and models. *Computers & Operations Research*, *132*, Article 105291. http://dx.doi.org/10.1016/j.cor.2021.105291, URL https://www.sciencedirect.com/science/article/pii/S0305054821000836.

Chen, J., Chu, C., Sahli, A., & Li, K. (2024). A branch-and-price algorithm for unrelated parallel machine scheduling with machine usage costs. *European Journal of Operational Research*, *316*(3), 856–872. http://dx.doi.org/10.1016/j.ejor.2024.03.011, URL https://www.sciencedirect.com/science/article/pii/S0377221724001851.

Chen, C., Fathi, M., Khakifirooz, M., & Wu, K. (2022). Hybrid tabu search algorithm for unrelated parallel machine scheduling in semiconductor fabs with setup times, job release, and expired times. *Computers & Industrial Engineering*, *165*, Article 107915. http://dx.doi.org/10.1016/j.cie.2021.107915, URL https://www.sciencedirect.com/science/article/pii/S0360835221008196.

Christofides, N., Alvarez-Valdes, R., & Tamarit, J. (1987). Project scheduling with resource constraints: A branch and bound approach. *European Journal of Operational Research*, *29*(3), 262–273. http://dx.doi.org/10.1016/0377-2217(87)90240-2, URL https://www.sciencedirect.com/science/article/pii/0377221787902402.

de Alba, H. G., Nucamendi-Guillén, S., & Avalos-Rosales, O. (2022). A mixed integer formulation and an efficient metaheuristic for the unrelated parallel machine scheduling problem: Total tardiness minimization. *EURO Journal on Computational Optimization*, *10*, Article 100034. http://dx.doi.org/10.1016/j.ejco.2022.100034, URL https://www.sciencedirect.com/science/article/pii/S2192440622000107.

Elyasi, M., Selcuk, Y. S., Özener, O. Ö., & Coban, E. (2024). Imperialist competitive algorithm for unrelated parallel machine scheduling with sequence-and-machine-dependent setups and compatibility and workload constraints. *Computers & Industrial Engineering*, *190*, Article 110086. http://dx.doi.org/10.1016/j.cie.2024.110086, URL https://www.sciencedirect.com/science/article/pii/S0360835224002079.

Ezugwu, A. E.-S. (2024). Metaheuristic optimization for sustainable unrelated parallel machine scheduling: A concise overview with a proof-of-concept study. *IEEE Access*, *12*, 3386–3416. http://dx.doi.org/10.1109/ACCESS.2023.3347047.

Fanjul-Peyro, L., Perea, F., & Ruiz, R. (2017). Models and matheuristics for the unrelated parallel machine scheduling problem with additional resources. *European Journal of Operational Research*, *260*(2), 482–493. http://dx.doi.org/10.1016/j.ejor.2017.01.002, URL https://www.sciencedirect.com/science/article/pii/S0377221717300073.

Fanjul-Peyro, L., Ruiz, R., & Perea, F. (2019). Reformulations and an exact algorithm for unrelated parallel machine scheduling problems with setup times. *Computers & Operations Research*, *101*, 173–182. http://dx.doi.org/10.1016/j.cor.2018.07.007, URL https://www.sciencedirect.com/science/article/pii/S0305054818301916.

Fomin, A., & Goldengorin, B. (2022). An exact algorithm for the preemptive single machine scheduling of equal-length jobs. *Computers & Operations Research*, *142*, Article 105742. http://dx.doi.org/10.1016/j.cor.2022.105742, URL https://www.sciencedirect.com/science/article/pii/S0305054822000417.

Fonseca, G. H., Figueiroa, G. B., & Toffolo, T. A. (2024). A fix-and-optimize heuristic for the Unrelated Parallel Machine Scheduling Problem. *Computers & Operations Research*, *163*, Article 106504. http://dx.doi.org/10.1016/j.cor.2023.106504, URL https://www.sciencedirect.com/science/article/pii/S0305054823003684.

Hasani, A., Hosseini, S. M. H., & Sana, S. S. (2022). Scheduling in a flexible flow shop with unrelated parallel machines and machine-dependent process stages: Trade-off between makespan and production costs. *Sustainability Analytics and Modeling*, *2*, Article 100010. http://dx.doi.org/10.1016/j.samod.2022.100010, URL https://www.sciencedirect.com/science/article/pii/S266725962200008X.

Jaykrishnan, G., & Levin, A. (2024). Scheduling with cardinality dependent unavailability periods. *European Journal of Operational Research*, *316*(2), 443–458. http://dx.doi.org/10.1016/j.ejor.2024.02.038, URL https://www.sciencedirect.com/science/article/pii/S037722172400170X.

Jiang, D., & Tan, J. (2016). Scheduling with job rejection and nonsimultaneous machine available time on unrelated parallel machines. *Theoretical Computer Science*, *616*, 94–99. http://dx.doi.org/10.1016/j.tcs.2015.12.020, URL https://www.sciencedirect.com/science/article/pii/S0304397515011810.

Joo, C. M., & Kim, B. S. (2015). Hybrid genetic algorithms with dispatching rules for unrelated parallel machine scheduling with setup time and production availability. *Computers & Industrial Engineering*, *85*, 102–109. http://dx.doi.org/10.1016/j.cie.2015.02.029, URL https://www.sciencedirect.com/science/article/pii/S0360835215001084.

Khadivi, M., Abbasi, M., Charter, T., & Najjaran, H. (2024). A mathematical model for simultaneous personnel shift planning and unrelated parallel machine scheduling. arXiv:2402.15670.

Kurt, A., & Cetinkaya, F. C. (2024). Unrelated parallel machine scheduling under machine availability and eligibility constraints to minimize the makespan of non-resumable jobs. *International Journal of Industrial Engineering and Management*, *15*(1), 18–33. http://dx.doi.org/10.24867/ijiem-2024-1-345, URL http://dx.doi.org/10.24867/IJIEM-2024-1-345.

Lei, D., & He, S. (2022). An adaptive artificial bee colony for unrelated parallel machine scheduling with additional resource and maintenance. *Expert Systems with Applications*, *205*, Article 117577. http://dx.doi.org/10.1016/j.eswa.2022.117577, URL https://www.sciencedirect.com/science/article/pii/S0957417422008909.

Lei, D., & Yang, H. (2022). Scheduling unrelated parallel machines with preventive maintenance and setup time: Multi-sub-colony artificial bee colony. *Applied Soft Computing*, *125*, Article 109154. http://dx.doi.org/10.1016/j.asoc.2022.109154, URL https://www.sciencedirect.com/science/article/pii/S1568494622004100.

Lian, X., Zheng, Z., Zhu, M., & Gao, X. (2024). Proactive scheduling for steel plants with unrelated parallel machines and time uncertainty. *Computers & Industrial Engineering*, *188*, Article 109890. http://dx.doi.org/10.1016/j.cie.2024.109890, URL https://www.sciencedirect.com/science/article/pii/S0360835224000111.

Muñoz-Díaz, M.-L., Escudero-Santana, A., & Lorenzo-Espejo, A. (2024). Solving an Unrelated Parallel Machines Scheduling Problem with machine- and job-dependent setups and precedence constraints considering Support Machines. *Computers & Operations Research*, *163*, Article 106511. http://dx.doi.org/10.1016/j.cor.2023.106511, URL https://www.sciencedirect.com/science/article/pii/S0305054823003751.

Orts, F., Puertas, A., Ortega, G., & Garzón, E. (2023). Quantum annealing solution for the unrelated parallel machine scheduling with priorities and delay of task switching on machines. *Future Generation Computer Systems*, *148*, 514–523. http://dx.doi.org/10.1016/j.future.2023.07.006, URL https://www.sciencedirect.com/science/article/pii/S0167739X23002583.

Paul, S. K., & Azeem, A. (2010). Selection of the optimal number of shifts in fuzzy environment: manufacturing company's facility application. *Journal of Industrial Engineering and Management*, *3*(1), URL https://raco.cat/index.php/JIEM/article/view/206084.

Perez Bernal, C., Salido, M. A., & March Moya, C. (2025). Optimizing energy efficiency in unrelated parallel machine scheduling problem through reinforcement learning. *Information Sciences*, *693*, Article 121674. http://dx.doi.org/10.1016/j.ins.2024.121674, URL https://www.sciencedirect.com/science/article/pii/S0020025524015883.

Perez-Gonzalez, P., Fernandez-Viagas, V., Zamora García, M., & Framinan, J. M. (2019). Constructive heuristics for the unrelated parallel machines scheduling

problem with machine eligibility and setup times. *Computers & Industrial Engineering*, *131*, 131–145. http://dx.doi.org/10.1016/j.cie.2019.03.034, URL https://www.sciencedirect.com/science/article/pii/S0360835219301706.

Ploydanai, K., & Mungwattana, A. (2010). Algorithm for solving job shop scheduling problem based on machine availability constraint. *International Journal on Computer Science and Engineering*, *2*.

Rabadi, G., Moraga, R., & Al-Salem, A. (2006). Heuristics for the unrelated parallel machine scheduling problem with setup times. *Journal of Intelligent Manufacturing*, *17*, 85–97. http://dx.doi.org/10.1007/s10845-005-5514-0.

Raboudi, H., Aalpan, G., Mangione, F., Tissot, G., & Noel, F. (2022). Scheduling unrelated parallel machines with a common server and sequence dependent setup times. *IFAC-PapersOnLine*, *55*(10), 2179–2184. http://dx.doi.org/10.1016/j.ifacol.2022.10.031, 10th IFAC Conference on Manufacturing Modelling, Management and Control MIM 2022. URL https://www.sciencedirect.com/science/article/pii/S2405896322020407.

Rolim, G. A., Nagano, M. S., & Prata, B. d. (2023). Formulations and an adaptive large neighborhood search for just-in-time scheduling of unrelated parallel machines with a common due window. *Computers & Operations Research*, *153*, Article 106159. http://dx.doi.org/10.1016/j.cor.2023.106159, URL https://www.sciencedirect.com/science/article/pii/S0305054823000230.

Sanati, H., Moslehi, G., & Reisi-Nafchi, M. (2023). Unrelated parallel machine energy-efficient scheduling considering sequence-dependent setup times and time-of-use electricity tariffs. *EURO Journal on Computational Optimization*, *11*, Article 100052. http://dx.doi.org/10.1016/j.ejco.2022.100052, URL https://www.sciencedirect.com/science/article/pii/S2192440622000284.

Santoro, M. C., & Junqueira, L. (2023). Unrelated parallel machine scheduling models with machine availability and eligibility constraints. *Computers & Industrial Engineering*, *179*, Article 109219. http://dx.doi.org/10.1016/j.cie.2023.109219, URL https://www.sciencedirect.com/science/article/pii/S0360835223002437.

Shahvari, O., & Logendran, R. (2017a). A bi-objective batch processing problem with dual-resources on unrelated-parallel machines. *Applied Soft Computing*, *61*, 174–192. http://dx.doi.org/10.1016/j.asoc.2017.08.014, URL https://www.sciencedirect.com/science/article/pii/S1568494617304969.

Shahvari, O., & Logendran, R. (2017b). An Enhanced tabu search algorithm to minimize a bi-criteria objective in batching and scheduling problems on unrelated-parallel machines with desired lower bounds on batch sizes. *Computers & Operations Research*, *77*, 154–176. http://dx.doi.org/10.1016/j.cor.2016.07.021, URL https://www.sciencedirect.com/science/article/pii/S0305054816301897.

Su, L. H., & Hsiao, M. C. (2015). Two-agent scheduling in open shops subject to machine availability and eligibility constraints. *Journal of Industrial Engineering Management*, *8*(4).

Wang, H., & Alidaee, B. (2019). Effective heuristic for large-scale unrelated parallel machines scheduling problems. *Omega*, *83*, 261–274. http://dx.doi.org/10.1016/j.omega.2018.07.005, URL https://www.sciencedirect.com/science/article/pii/S0305048318302081.

Wang, B., Feng, K., & Wang, X. (2023). Bi-objective scenario-guided swarm intelligent algorithms based on reinforcement learning for robust unrelated parallel machines scheduling with setup times. *Swarm and Evolutionary Computation*, *80*, Article 101321. http://dx.doi.org/10.1016/j.swevo.2023.101321, URL https://www.sciencedirect.com/science/article/pii/S2210650223000949.

Wang, H., Li, R., & Gong, W. (2023). Minimizing tardiness and makespan for distributed heterogeneous unrelated parallel machine scheduling by knowledge and Pareto-based memetic algorithm. *Egyptian Informatics Journal*, *24*(3), Article 100383. http://dx.doi.org/10.1016/j.eij.2023.05.008, URL https://www.sciencedirect.com/science/article/pii/S1110866523000312.

Wang, J., Song, G., Liang, Z., Demeulemeester, E., Hu, X., & Liu, J. (2023). Unrelated parallel machine scheduling with multiple time windows: An application to earth observation satellite scheduling. *Computers & Operations Research*, *149*, Article 106010. http://dx.doi.org/10.1016/j.cor.2022.106010, URL https://www.sciencedirect.com/science/article/pii/S0305054822002428.

Wang, S., Wu, R., Chu, F., & Yu, J. (2022). Unrelated parallel machine scheduling problem with special controllable processing times and setups. *Computers & Operations Research*, *148*, Article 105990. http://dx.doi.org/10.1016/j.cor.2022.105990, URL https://www.sciencedirect.com/science/article/pii/S030505482200226X.

Wang, S., Wu, R., Chu, F., & Yu, J. (2023). An exact decomposition method for unrelated parallel machine scheduling with order acceptance and setup times. *Computers & Industrial Engineering*, *175*, Article 108899. http://dx.doi.org/10.1016/j.cie.2022.108899, URL https://www.sciencedirect.com/science/article/pii/S0360835222008877.

Wang, M., Zhang, J., Zhang, P., Xiang, W., Jin, M., & Li, H. (2024). Generative deep reinforcement learning method for dynamic parallel machines scheduling with adaptive maintenance activities. *Journal of Manufacturing Systems*, *77*, 946–961. http://dx.doi.org/10.1016/j.jmsy.2024.11.004, URL https://www.sciencedirect.com/science/article/pii/S0278612524002589.

Wu, H., & Zheng, H. (2023). Single-machine scheduling with periodic maintenance and learning effect. *Scientific Reports*, *13*, http://dx.doi.org/10.1038/s41598-023-36056-w.

Yepes-Borrero, J. C., Villa, F., Perea, F., & Caballero-Villalobos, J. P. (2020). GRASP algorithm for the unrelated parallel machine scheduling problem with setup times and additional resources. *Expert Systems with Applications*, *141*, Article 112959. http://dx.doi.org/10.1016/j.eswa.2019.112959, URL https://www.sciencedirect.com/science/article/pii/S0957417419306773.

Ying, K. C., Pourhejazy, P., & Huang, X. Y. (2024). Revisiting the development trajectory of parallel machine scheduling. *Computers & Operations Research*, *168*, Article 106709. http://dx.doi.org/10.1016/j.cor.2024.106709, URL https://www.sciencedirect.com/science/article/pii/S0305054824001813.

Yizhuo Zhu, D. L. (2024). Dynamical artificial bee colony for energy-efficient unrelated parallel machine scheduling with additional resources and maintenance. *Computers, Materials & Continua*, *81*(1), 843–866. http://dx.doi.org/10.32604/cmc.2024.054473, URL http://www.techscience.com/cmc/v81n1/58314.

Zampella, M., Otamendi, U., Belaunzaran, X., Artetxe, A., Olaizola, I. G., Sierra, B., et al. (2025). Exploring multi-agent reinforcement learning for unrelated parallel machine scheduling. *Journal of Supercomputing*, *81*(4), http://dx.doi.org/10.1007/s11227-025-07030-2.

Zheng, F., Jin, K., Xu, Y., & Liu, M. (2022). Unrelated parallel machine scheduling with processing cost, machine eligibility and order splitting. *Computers & Industrial Engineering*, *171*, Article 108483. http://dx.doi.org/10.1016/j.cie.2022.108483, URL https://www.sciencedirect.com/science/article/pii/S0360835222005058.

Alejandro Uribe Alejandro Uribe has a bachelor's degree in Physics Engineering and a Master's degree in Engineering from EAFIT University. He is currently studying his Mathematical Engineering PhD at the same university and working as a research assistant at the Foundation Center for Visual Interaction and Communications Technologies, Vicomtech. His studies have focused on simulation and optimization using Python programming tools. He is interested in mathematical problem formulation, heuristics, renewable energies, artificial intelligence, and new technologies.

Urtzi Otamendi Urtzi Otamendi has a bachelor's degree in Computer Engineering from the Faculty of Computer Science of the Public University of the Basque Country. He completed his Master's Degree in Artificial Intelligence at the Polytechnic University of Madrid. He works as a research assistant in the Foundation Center for Visual Interaction and Communications Technologies, Vicomtech, in the Data Intelligence for Energy and Industrial Processes department. His interests include software development, artificial intelligence, and computer vision applied to innovative technologies. Passionate about advancing intelligent systems for real-world applications.

Arkaitz Artetxe Arkaitz Artetxe holds a PhD in Computer Science and a Master's in Computational Engineering and Intelligent Systems, both from the University of the Basque Country. He is a researcher at Vicomtech, where he has worked in e-health and biomedical applications as well as in data intelligence for energy and industrial processes. His work focuses on applying computational engineering and data mining to real-world applications. He has published and reviewed extensively in high-impact international conferences and journals.

Juan Carlos Rivera Juan Carlos Rivera holds a Bachelor's degree in Industrial Engineering and a Master's in Computer Science from the National University of Colombia (Medellin campus). He earned a Ph.D. in System Optimization and Reliability from the University of Technology of Troyes (France). Currently, he is a Full Professor at EAFIT University in the field of Computing and Analytics. His main research interests focus on the development of mixed-integer linear programming models and metaheuristic strategies to solve optimization problems, with applications in logistics and production.